



LIFEBOOK
Technical Requirements Document

LifeBook

System architecture, and what automated testing does not tell you

Prepared by LifeBook Inc. • Version 4.0 • September 2026

1. Architecture philosophy

The backend is a set of single-purpose agents coordinated by an orchestrator, built with zero runtime dependencies beyond firebase-admin. Every route resolves identity through one function; every check-in passes through retrieval before generation; every generation passes through review before the client sees it.

Important scope note: every claim of "tested" in this document means verified by direct API calls during development — confirming the code does what it was written to do. It does not mean confirmed by real user behavior. See the PRD, section 2, for that distinction.

2. Agent & module inventory

Module	Responsibility
orchestrator.ts	Sequences retrieval → generation → review, persists results, updates streaks
verseRetrieval.ts	Selects from a fixed, vetted 12-verse corpus tagged to the 6-mood set
contentGeneration.ts	Writes reflection/prayer/action step around the given verse; dev-fallback template text when no API key is set
safetyReview.ts	Keyword-based distress detection on free-text notes
communityModeration.ts	First-pass screening on prayer requests and discussion posts
journeys.ts	Journey browsing, starting, advancement, and mood-based recommendation scoring
badges.ts	Computes badges from real streak data
journal.ts	Journal entries, favorites, mood-history
notifications.ts	Push token storage and sending through Expo's push service
auth/verifyToken.ts	Resolves request identity — verified Firebase uid when configured, deviceId otherwise
security/cors.ts, security/rateLimit.ts	Origin allow-list and in-memory per-IP rate limiting

3. Technology stack

Layer	Choice
Frontend	React Native / Expo (managed workflow)
Backend	Node.js + TypeScript, zero-dependency HTTP server run via tsx
Authentication	Firebase Auth (email/password), dev-fallback to anonymous id
Database	JSON file store — explicitly a bridge, not the production destination

Layer	Choice
AI engine	Claude (Anthropic API); dev-fallback template text when no key is set
Push notifications	Expo push service, not a direct FCM/APNs call
Deployment	Dockerfile + Render Blueprint

4. What this backend cannot currently tell you

- Where in the guided flow real users drop off — no analytics events are fired anywhere in the mobile app today.
- Whether the AI-generated content is theologically sound at scale — the review queue exists, but nothing currently populates it except manual test flags.
- Whether the rate limits are set to sensible real-world values — defaults (60/min general, 10/min check-in) are reasonable guesses, not tuned against real traffic.

Adding basic event instrumentation (screen views, flow step completion, drop-off) is a small, concrete engineering task that would directly serve the PRD's validation priority — worth doing before the next round of feature work, not after.

5. Security

- CORS: tested across all four cases (open by default, blocked when disallowed, correctly matched when allowed, unaffected by requests with no Origin header).
- Rate limiting: tested live — a lowered threshold correctly allowed N requests and 429'd the next, with a retryAfterMs.
- Known limitation: rate limiter state is in-memory per process, not a hard cap across multiple horizontally-scaled instances.

6. Deployment

A Dockerfile and Render Blueprint exist and were verified locally (health checks, configurable persistent storage via DATA_DIR). Neither has been deployed to a live URL. Per the PRD's validation priority, deployment should follow successful small-scale user testing, not precede it — there is little value in a public URL for a product that hasn't been shown to work for anyone yet.